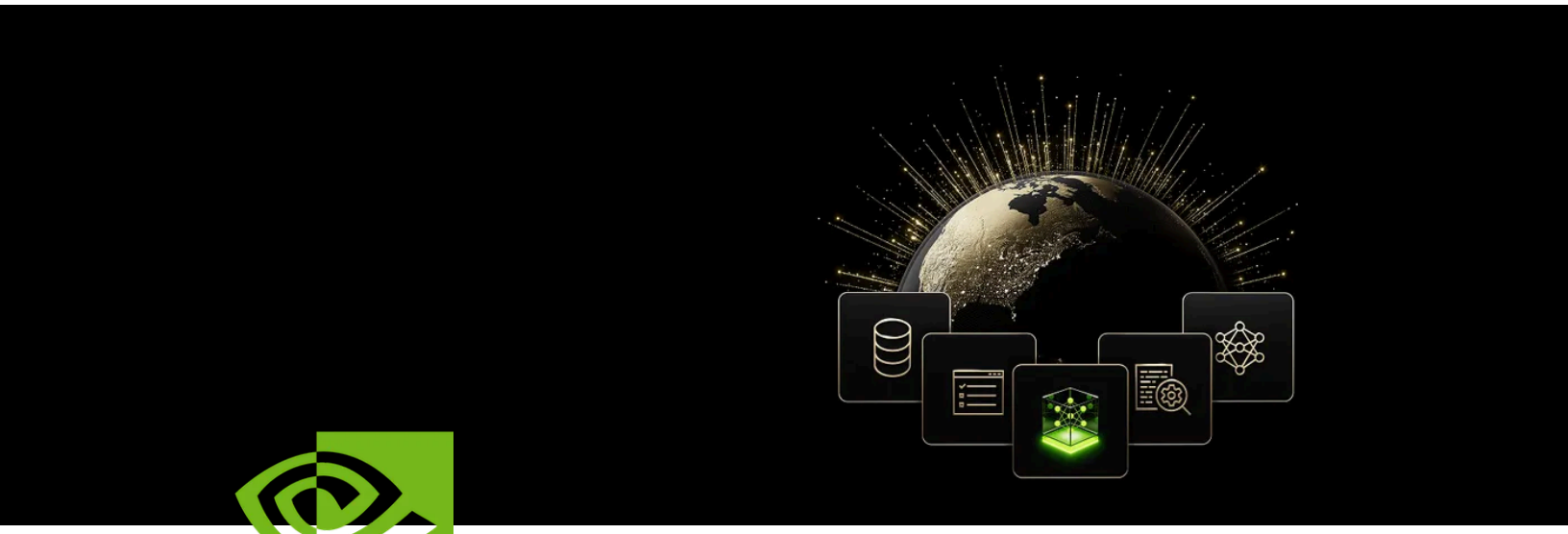


NVIDIA x Red Hat Small Models Hack

October 17–18, 2026

In Person

Red Hat Annex 190 Davie St, Raleigh, NC, 27601

🚀 Project Submissions

Submission Link:

https://airtable.com/appm7NS9oN8K3friP/pagqXe6EIIlXx6oa3/form

Demo Video Instructions:

📺 Demo Video Instructions

Submission Checklist:

✅ Submission Checklist

🏁 Resources

- Ask questions, communicate with our team, connect with other hackers on the #small-models-hack-raleigh Discord channel
- **Wifi**: Once checking in at the front, you should've received an email with the subject line: "Your personal passphrase for Red Hat Wi-Fi". Please use that to connect to the network.

⁉️ If you have any questions, please reach out in the #small-models-hack-raleigh channel in the NVIDIA Developer Discord server.

💪 The Challenge + Bounties

Welcome to the Small Models Hack, Hosted by NVIDIA and Red Hat!
We're excited to host a diverse group of builders for a weekend.

Challenge Rules

- **Total parameters must be ≤ 33 billion** (applies to the total params of all generative language models, including any used for verification, critique, routing, or speculative decoding. Embedding and reranking models are excluded). Models must have 33 billion or fewer total parameters, based on the official parameter count published in the model developer's model card. For MoE models, total parameters—not just active parameters—determine eligibility.
- All submissions **must align with one of the three challenge tracks below** and **meaningfully leverage open data**.
  - **Existing Open Data:** Teams may use publicly available datasets or open-data APIs.
  - **Synthetic Data:** Teams may generate synthetic datasets during the hackathon, provided they publish the resulting data and methodology openly.
  - **Across Tracks:** Open data can be used for fine-tuning, multi-agent workflows, or tool-calling applications.
  - **Documentation:** Teams must reference their data sources and demonstrate how open data is incorporated into their solution.
  - **Flexibility:** Each track defines how your AI works, not what problem it solves—teams are free to tackle any real-world challenge through the lens of their chosen track.
- All code required to run **MUST be written during the hackathon**. Open source code is allowed only if it's been open-sourced for 2+ weeks.

Main Challenge Tracks

🎯 1. Fine-Tuned & Specialized

Focus: A small model that's excellent at one real thing.

The Goal: Adapt or fine-tune a small model for a specific job—then publish it openly so others can reuse it. Depth over breadth, and evidence that specialization beats general-purpose for your chosen task.

🕸️ 2. Multi-Agent Systems

Focus: Many small models beating one big one.

The Goal: Wire several small models together so they coordinate—dividing up work, passing context back and forth, building on each other's output—to accomplish something no single model could alone.

🛠️ 3. Tool-Calling

Focus: A single model that uses real tools reliably.

The Goal: Deciding when to call, picking the right tool, formatting valid arguments, handling what comes back, recovering from failed or malformed calls, verifying its own work, and falling back gracefully when something breaks.

💰 Additional Bounties

Best Use of NVIDIA Nemotron Lighting

Best Use of NVIDIA Nemotron Lightning

ɜest Use of NVIDIA lemotron Lightning

ɹunty

Challenge: Build a project where Nemotron Lightning plays a meaningful role in doing real work. Use it on its own or alongside other models, & show what it enables.

Required: Use the Nemotron Lightning model as a core part of the project.

Demonstrate: Show Nemotron Lightning's contribution to the overall experience, such as quality, speed, efficiency, usefulness, or creativity.

Submit: A short explanation of what Nemotron Lightning is doing, why you chose it for that part of the project, and how you are optimizing its capabilities.

PRIZES: NVIDIA RTX 5070 GPU
NVIDIA Developer Livestream Feature
Social Amplification



Best Model Improvement With NVIDIA NeMo Gym

ɜest Model Improvement Vith NeMo Gym

ɹunty

Challenge: Pick one task a small model or agent fails at today, and make the score better with NeMo Gym.

Required: A NeMo Gym environment with tasks, tools the agent can use, and code that scores each attempt.

Demonstrate: Baseline vs. final scores on the same held-out tasks and eval configuration. Isolate the improvements that made it.

Submit: Your NeMo Gym setup, baseline vs. final results, how to run scoring, and a short explanation of how you made improvements.

PRIZES: NVIDIA RTX 5070 GPU
NVIDIA Developer Livestream Feature
Social Amplification



Best Use of Red Hat vLLM

ɛst Use of Red Hat vLLM

ɹunty

Challenge: Build a project that is genuinely served on vLLM. vLLM should play a meaningful role in how your application runs, not just be added at the end.

Required: Use vLLM as the serving layer for a core model in your project. Show that your serving setup is functional and central to the build.

Demonstrate: Show how you used vLLM and what you got out of it – such as serving performance, efficiency, throughput, latency, scalability, or reliability.

Submit: Briefly explain how vLLM fits into your architecture, how you configured and optimized it, any measurable results. Depth over breadth.

PRIZES: JBL Speaker (per team member)
Red Hat Office Hours Feature
Social Amplification



NVIDIA Developer Champions Choice Award

∀IDIA Champions hoice Award

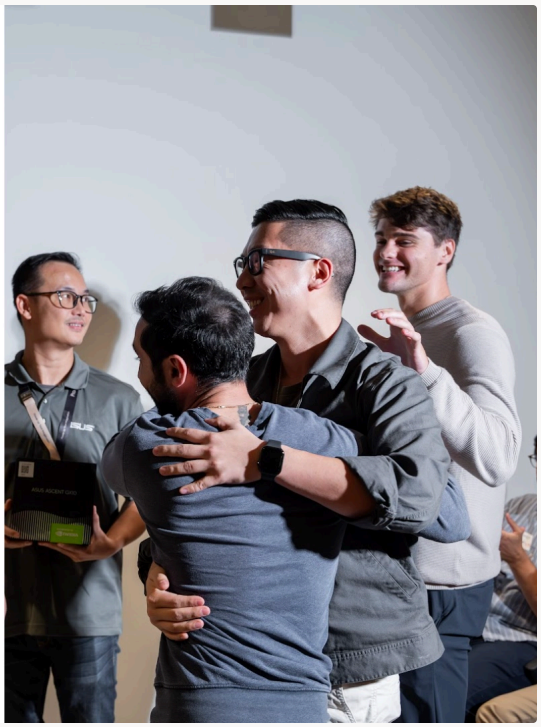
ɹnty

ɛlect group of NVIDIA Developer Champions will ɹd this bounty to their favorite project!

PRIZE:
$100 NVIDIA Brev Credits per team member!

ɓout NVIDIA Developer Champions Program

select cohort of community leaders and ɜssionate builders who lead, teach, and mentor, ɾringing their developer community along with them



Redeeming Brev Credits

NVIDIA Brev Getting Started Overview

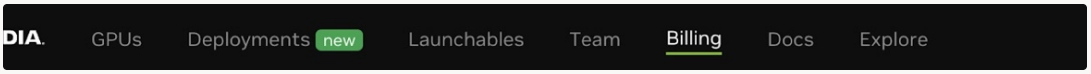
NVIDIA Brev Documentation | NVIDIA Brev Documentation
Get started with NVIDIA Brev - GPU-accelerated AI/ML development platform. Access configured GPU instances across multiple cloud providers.
https://docs.nvidia.com/brev/getting-started/overview

Adding Users to the Instance — Steps For Team Captains

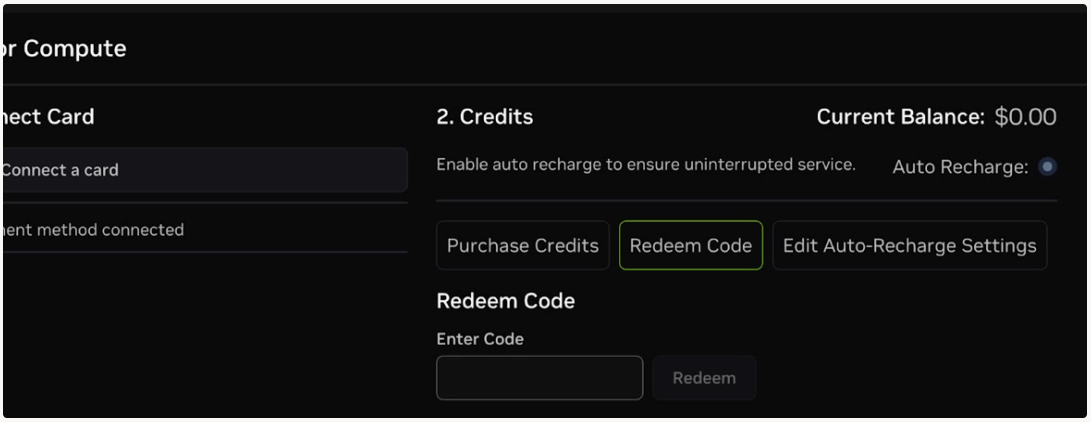
Team captains will be in charge of inviting their teammates to their Brev Org after redeeming the coupon code.

Step 1: Please go to http://brev.nvidia.com and input your email to create an account.

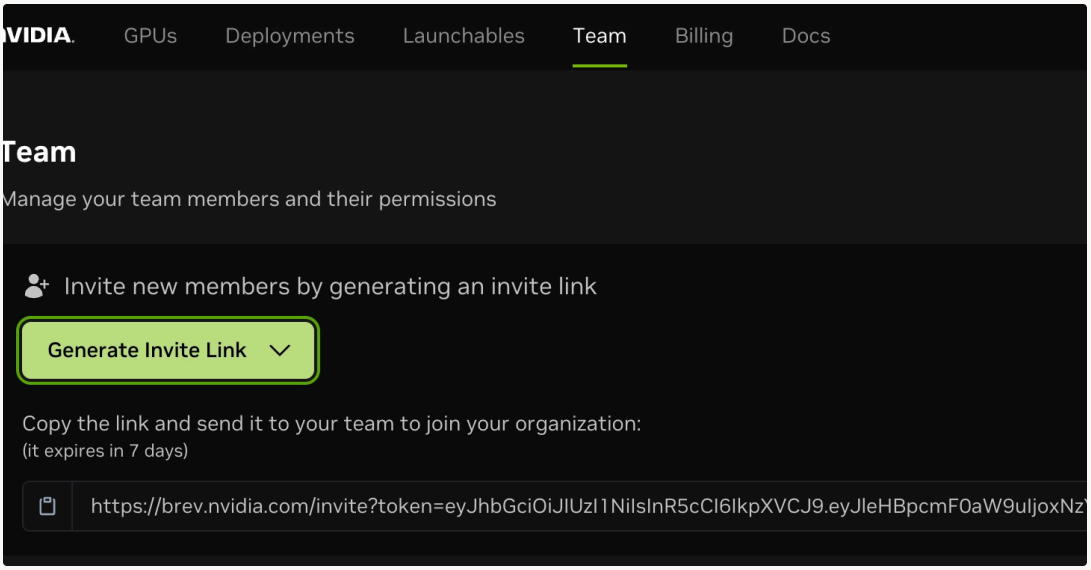
Step 2: Once you have signed up, please go to the 'Billing' tab located at the top of the Brev console (as shown in the screenshot below).



Step 3: Once you have navigated to the ‘Billing’ tab, scroll down and click 'Redeem Code'. Then, enter the code you were provided in the 'Enter Code' field and click 'Redeem.'



Step 4: From the "**Team**" tab of the Brev Console please click "**Generate Invite Link**" and share with your Team. (These invite link expire after 7 days, but the team leader can generate new invite links as needed in case a user forgets to click with 7 days)

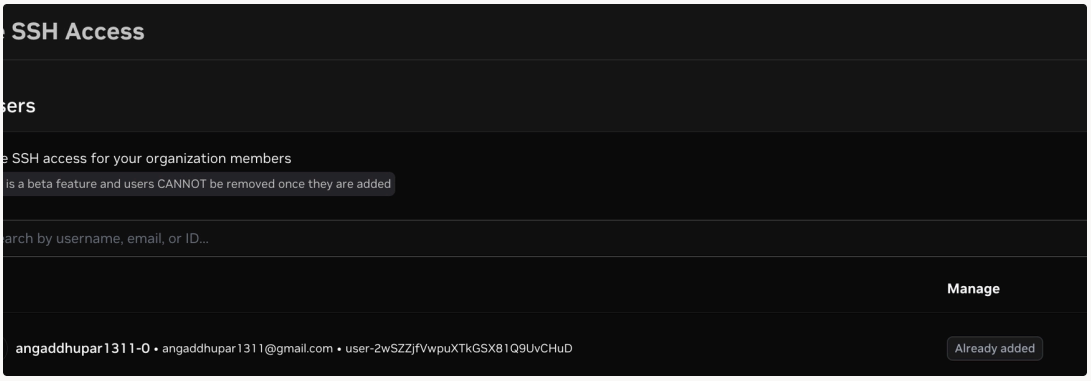


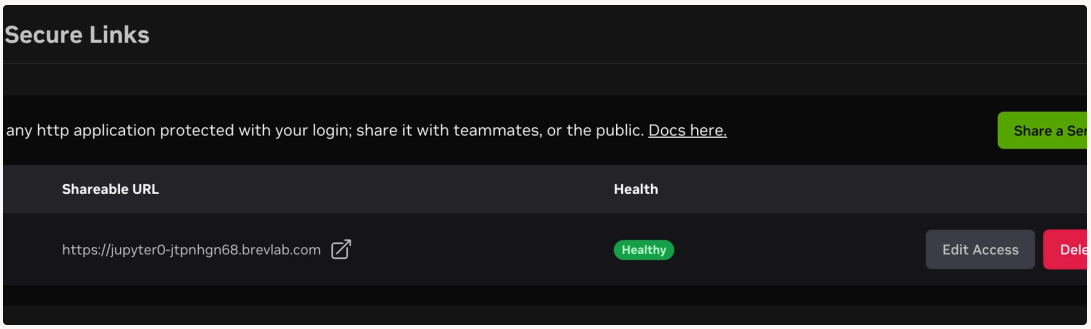
5) Steps to add your teammates to your GPU Instance and give them SSH access + access to the Jupyter Notebook:

Users first need to be added to the Org to get access to the instance. To add a user to your Brev org - you need to navigate to the 'Team' tab and click 'Generate Invite link".

The user who created the instance, will need navigate to the "GPU" tab, then click on the instance you want to share, scroll all the way down and need to the "Share SSH Access" section of the instance and add them. *(*Instance will need to be running to share SSH access)*

If your teammate also needs access to the jupyter notebook, please make sure to add their email or username to the "Using secure links" section by clicking the **edit access button** -> **add username or email** -> click "**share**". *(*Container will need to be full built before sharing SSH Access).*





‼️ **Still Need Help?** Please reach out to brev-support@nvidia.com with any issues

⚖️ Judging

Philosophy

We are judging **Systems Engineering**. A winning project isn't just a slide deck or a simple API wrapper; it's a **working, end-to-end system** that meaningfully uses small models to process information, take action, or solve a real-world problem.

1. Technical Execution & Completeness

Did they actually build a working, complex system?

- **Completeness:** Does the solution successfully execute its intended workflow end-to-end without major failures? Does it meaningfully leverage open data?
- **Technical Depth:** Is there significant engineering "under the hood"? Did they implement meaningful model adaptation, multi-agent orchestration, robust tool-calling, or custom logic rather than just a basic API wrapper?
    - **Fine-Tuned & Specialized:** Did they successfully adapt a small model for a specific task? Can they demonstrate measurable improvement over a baseline using evaluation data?
    - **Multi-Agent Systems:** Do multiple small models meaningfully collaborate, divide responsibilities, and exchange information to accomplish a task beyond what a single model could easily achieve?
    - **Tool-Calling:** Can a single small model reliably select and execute tools, handle errors, and recover from failures rather than simply follow a hardcoded sequence of calls?